DAY 10 | POST-READ

Async JavaScript and Express: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Asynchronous	Start slow work, keep running other code, handle the result later.
Event loop	One thread runs your code; finished background work queues callbacks.
Promise	pending, then fulfilled (value) or rejected (error). .then / .catch / .finally.
async / await	async functions return promises; await pauses that function only. try/catch for errors.
Promise.all	Run independent tasks in parallel; rejects if any one fails.
fetch	await fetch(url); check res.ok; await res.json().
npm	npm init -y, npm install pkg, package.json, never commit node_modules.
Express	express(), app.use(express.json()), app.get/post/put/delete, res.status().json().
Request data	req.params (path), req.query (query string), req.body (JSON).
Postman	Requests, JSON bodies, collections and {{baseUrl}} environments.

Part B: Going beyond today

1. Promise.allSettled and Promise.race

```
const results = await Promise.allSettled([taskA(), taskB()]);
results.forEach(r => console.log(r.status));   // fulfilled or rejected
```

allSettled never fails fast; race returns whichever finishes first, useful for timeouts.

2. Express routers

```
const router = express.Router();
router.get("/", listTasks);
app.use("/tasks", router);
```

The same idea as FastAPI's APIRouter: one file per resource.

3. Error handling middleware

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: "Something went wrong" });
});
```

4. nodemon for auto reload

npm install --save-dev nodemon, then add "dev": "nodemon server.js" to scripts. It restarts the server on every save, like uvicorn --reload.

5. Environment variables

```
const PORT = process.env.PORT || 3000;
```

6. Postman tests

```
pm.test("status is 201", () => pm.response.to.have.status(201));
pm.test("has id", () => pm.expect(pm.response.json().id).to.exist);
```

Postman can run these checks for a whole collection with one click, a quick regression suite.

Part C: Check yourself

- Why does setTimeout(fn, 0) still run after the next console.log?
- What are the three states of a promise?
- What is the difference between awaiting two calls one after another and using Promise.all?
- Where do req.params, req.query and req.body come from?
- Why should node_modules not be committed to Git?

Part D: Practice ideas for the weekend

- Add PUT and filtering by done=true or false to your Express Tasks API.
- Write a client script that creates 5 tasks in parallel with Promise.all and then lists them.
- Add Postman tests to every request in your collection and run the whole collection.

Coming up on Day 11: React basics: single page apps, JSX, components, props, state and events. There is also an assessment (MCQ and coding).

Yoi shuumatsu wo (良い週末を): have a great weekend